

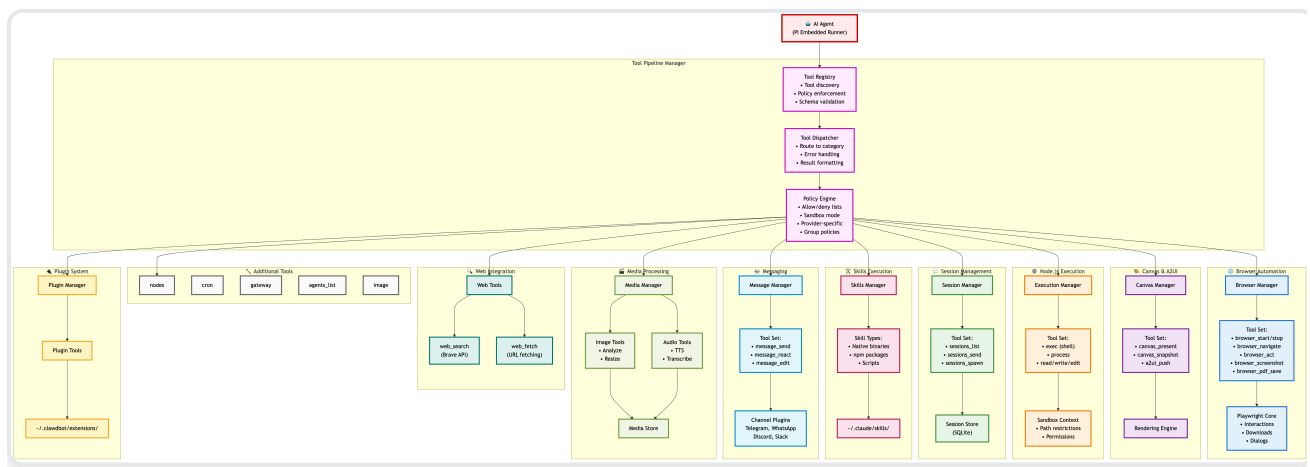


Clawdbot - Tool Pipeline Architecture

Table of Contents

1. [Tool Pipeline Diagram](#)
2. [Overview](#)
3. [Pipeline Manager](#)
4. [Browser Automation](#)
5. [Canvas & A2UI](#)
6. [Node.js Execution](#)
7. [Session Management](#)
8. [Skills Framework](#)
9. [Web Integration](#)
10. [Messaging](#)
11. [Media Processing](#)
12. [Additional Tools](#)
13. [Plugin System](#)
14. [Execution Flow](#)
15. [Security Features](#)
16. [Summary](#)

Tool Pipeline Architecture Diagram



Overview

The Tool Pipeline is the execution layer that enables the AI agent to interact with external systems, execute commands, manipulate files, and perform various operations. It follows a layered architecture with strict policy enforcement and supports over 50 different tools across 10+ categories.

1. Tool Pipeline Manager

Purpose: Central coordination and routing for all tool executions

Tool Registry

Discovers and registers all available tools

- Scans built-in tools
- Loads plugin tools
- Validates tool schemas (TypeBox/Zod)
- Maintains tool metadata

Tool Dispatcher

Routes tool calls to appropriate handlers

- Parses tool call parameters
- Validates input schemas
- Handles execution errors
- Formats results for agent consumption

Policy Engine

Enforces access control and security

- Allow/deny list evaluation (5 layers)
- Sandbox mode restrictions
- Provider-specific tool availability
- Group-level policies
- Per-agent policies

2. Browser Automation

Purpose: Web browser control and automation via Playwright

Tools: browser_start, browser_stop, browser_status, browser_navigate, browser_act, browser_screenshot, browser_pdf_save, browser_console, browser_tabs, browser_open_tab, browser_close_tab, browser_focus_tab

Core Capabilities

- Start/stop browser instances
- Navigate to URLs
- Interact with elements
- Take screenshots

Advanced Features

- Save pages as PDF
- Monitor console
- Handle file downloads
- Manage dialogs

Implementation:

- **Playwright Core:** Low-level browser control
- **Node Proxy:** Distributed browser instances on remote nodes
- **Browser Profiles:** Multiple contexts (logged-in states)

Key Files: src/agents/tools/browser-tool.ts , src/browser/pw-tools-core.ts

3. Canvas & A2UI

Purpose: Visual UI rendering and Agent-to-UI communication

Tools: canvas_present, canvas_hide, canvas_navigate, canvas_eval, canvas_snapshot, a2ui_push, a2ui_reset

Use Cases:

- Visual feedback to users
- Interactive dashboards
- Data visualization
- UI prototyping

Key Files: src/agents/tools/canvas-tool.ts

4. Node.js Execution Environment

Purpose: Execute shell commands and manage file operations

Tools: exec, process, read, write, edit, apply_patch

Security Features

- **Sandbox Context:** Isolated workspace per agent
- **Path Restrictions:** Prevent access outside workspace
- **Safe Binary Allowlist:** Whitelist of allowed commands
- **Timeout Control:** Prevent runaway processes
- **Approval System:** User confirmation for dangerous operations

Key Files: `src/agents/bash-tools.ts` , `src/agents/sandbox.ts`

5. Session Management

Purpose: Manage conversation sessions and inter-agent communication

Tools: `sessions_list`, `sessions_history`, `sessions_send`, `sessions_spawn`, `session_status`

Architecture:

- **Session Store:** SQLite database for persistence
- **Routing Engine:** Maps channels to agents
- **Session Keys:** Unique identifiers (e.g., `telegram:user:123:main`)

Use Cases: Multi-agent workflows, task delegation, conversation context management

Key Files: `src/agents/tools/sessions-*.ts` , `src/routing/session-key.ts`

6. Skills Execution Framework

Purpose: Execute external skills (CLI tools, scripts, packages)

Skill Types:

- Native binaries (e.g., `gog` , `pandoc`)
- npm packages
- Shell scripts
- Python scripts

Workflow: 1. Skill discovery in `~/.claude/skills/` 2. Dependency installation (if needed) 3. Command construction from `SKILL.md` 4. Execution with parameters 5. Result parsing and return

Skill Location: `~/.claude/skills/skill-name/`

7. Web Integration

Purpose: Web search and content fetching

Tools: web_search, web_fetch

web_search

- Brave Search API
- Query processing
- Result ranking
- Context extraction

web_fetch

- URL fetching
- HTML parsing
- Content extraction
- Media downloading

Features: 15-minute response cache, deduplication, sandbox mode support

Key Files: `src/agents/tools/web-tools.ts`

8. Messaging

Purpose: Send messages through various channels

Tools: message (send), message_react, message_edit, message_delete

Supported Channels: Telegram, WhatsApp, Discord, Slack

Features:

- **Thread Management:** Reply-to linking, topic routing
- **Auto-threading:** Automatic thread continuation (Slack)
- **Rich Media:** Images, audio, video attachments

Key Files: `src/agents/tools/message-tool.ts` , `src/channels/plugins/`

9. Media Processing

Purpose: Process images, audio, and video

Image Processing

- Vision analysis (LLM)
- Resize and crop
- Format conversion
- Metadata extraction

Audio Processing

- Text-to-speech
- Transcription (future)
- Format conversion

Media Store: Local filesystem storage with metadata tracking and automatic cleanup

Key Files: `src/agents/tools/image-tool.ts` , `src/agents/tools/tts-tool.ts`

10. Additional Tools

Tool	Purpose	Key Features
nodes	Manage remote nodes	Discovery, capabilities, health monitoring
cron	Schedule tasks	Cron expressions, timezone support
gateway	Gateway control	Configuration, system status, admin ops
agents_list	List agents	Metadata, capabilities, configuration
image	Vision analysis	Multi-modal input, OCR capabilities

11. Plugin System

Purpose: Extend tool capabilities via plugins

Features:

- Dynamic tool registration
- Custom tool implementation
- Runtime isolation
- Manifest-based discovery

Plugin Location: `~/.clawdbot/extensions/`

Key Files: `src/plugins/tools.ts` , `src/plugins/registry.ts`

Tool Execution Flow

1. Agent Request ↓ 2. Tool Registry → Validate tool exists ↓ 3. Policy Engine → Check permissions (5 layers) ↓ 4. Tool Dispatcher → Route to handler ↓ 5. Tool Implementation → Execute ↓ 6. Result Formatting → Return to agent

Policy Enforcement Layers

1. **Global Policy:** `config.tools.allow`
2. **Provider Policy:** `config.agents.defaults.models[provider].tools.allow`
3. **Agent Policy:** `config.agents.list[agent].tools.allow`
4. **Group Policy:** Channel/group-specific restrictions
5. **Subagent Policy:** Inherited from parent + restrictions

Evaluation: Most restrictive policy wins

Security Features

- **Sandbox Mode:** Isolated workspace, restricted tool access
- **Path Restrictions:** Tools cannot access files outside workspace
- **Safe Binary Allowlist:** Only approved commands can be executed
- **Timeout Control:** Prevent long-running operations
- **Approval System:** User confirmation for risky operations
- **Plugin Isolation:** Plugins run in separate context
- **Schema Validation:** TypeBox/Zod validation for all inputs

Tool Categories Summary

Category	Tools	Key Features
Browser	12+ actions	Playwright automation, screenshots, PDF
Canvas	7 actions	UI rendering, A2UI, snapshots
Execution	6 tools	Shell, files, background processes
Sessions	5 tools	Multi-agent, spawning, messaging
Skills	Dynamic	External CLI tools, scripts
Web	2 tools	Search, fetch, caching
Messaging	4+ tools	Multi-channel, threading
Media	3 categories	Image, audio, video processing
Additional	5 tools	Nodes, cron, gateway, agents, vision
Plugins	Dynamic	Custom extensions

Total Tool Categories: 10+

Total Tools: 50+

Architecture Version: 1.0

Last Updated: 2026-02-07

Document Type: Tool Pipeline Architecture